

NYCU-EE IC LAB – Fall 2026

Lab02 Exercise

Design: Low Density Parity Check Code Decoder (LDPC)

Data Preparation

1. Extract test data from TA's directory:
`% tar -xvf ~iclabTA01/Lab02_2026F.tar`
2. The extracted Lab02/ directory contains:
 - a. 00_TESTBED
 - b. 01_RTL
 - c. 02_SYN
 - d. 03_GATE
 - e. 09_SUBMIT

Preliminary

● LDPC Codes

Low Density Parity Check (LDPC) codes are a class of linear block codes that can approach the Shannon limit. They are widely adopted in modern communication systems.

LDPC codes can be defined by the sparse parity check matrix $\mathbf{H}$ of size $m \times n$. A sequence $\mathbf{x}$ is a valid codeword if and only if $\mathbf{H}\mathbf{x}^T = \mathbf{0}$. The parity check matrix can also be represented by a bipartite graph, called a Tanner graph. As shown in Fig. 1, the Tanner graph consists of m check nodes (CNs) and n variable nodes (VNs). Each 1 in $\mathbf{H}$ represents an edge between the corresponding CN and VN.

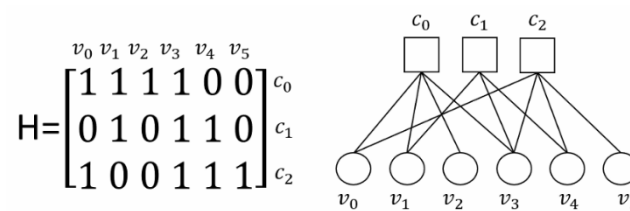


Fig.1

● Quasi-Cyclic (QC) LDPC Codes

The parity-check matrix of a QC-LDPC code consists of circulant permutation matrices (CPMs) and zero matrices. A base matrix provides a compact representation of this structure: each entry specifies the cyclic shift of the corresponding CPM. Fig. 2 shows an example of a base graph and its expanded parity check matrix. For example, an entry of 2 indicates that the identity matrix is cyclically shifted to the right by two columns, an entry of 0 represents the identity matrix, and an entry of -1 represents an all-zero submatrix.

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Fig.2

- **Message Passing Decoding**

LDPC decoders are commonly implemented using message passing decoding. As illustrated in Fig. 3, messages are exchanged iteratively between VNs and CNs to estimate the transmitted codeword bits. These messages carry extrinsic information; that is, the message sent from one node to another excludes the information originally received from that target node. The decoding process continues until the syndrome check passes or the maximum number of iterations is reached.

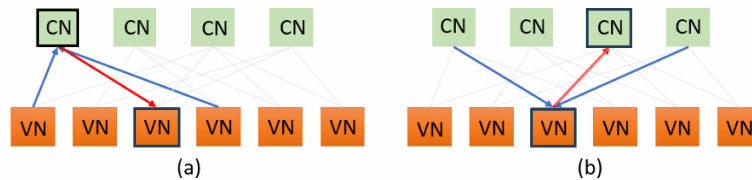


Fig.3

Design Description

- **LDPC Code Specification**

In this lab, you will design a decoder for the QC-LDPC code defined by the base graph below:

$$H_{BG} = \begin{bmatrix} -1 & 14 & 10 & 2 & 13 & 12 & 9 & 3 \\ 5 & -1 & 14 & 10 & 2 & 13 & 12 & 9 \\ 0 & 5 & -1 & 14 & 10 & 2 & 13 & 12 \\ 7 & 0 & 5 & -1 & 14 & 10 & 2 & 13 \end{bmatrix}$$

The base graph has 4 row groups and 8 column groups. Each entry is expanded into either a 16×16 circulant permutation matrix (CPM) or a 16×16 zero matrix, resulting in a 64×128 parity-check matrix H .

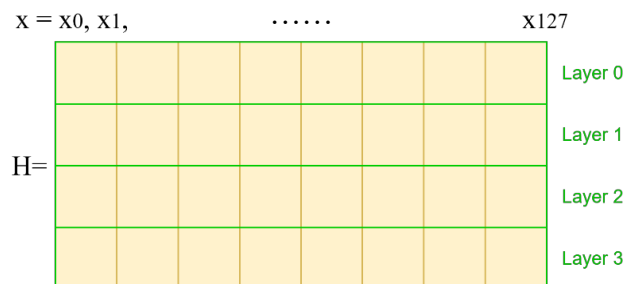


Fig.4

For layered scheduling, $\mathbf{H}$ is divided into 4 layers, where each layer corresponds to one CPM row block, as shown in Fig. 4.

● Input

The **in_data_valid** signal remains high for 128 consecutive cycles, during which the decoder receives one quantized channel log-likelihood ratio (LLR) per cycle, corresponding to codeword bits x_0 to x_{127} . After all 128 channel LLRs are received, the decoder starts the normalized min-sum (NMS) decoding algorithm according to the decoding schedule selected by **in_mode**.

$$\text{channel LLR of } x_j = \ln \frac{p(x_j = 0 \mid j^{\text{th}} \text{ received value})}{p(x_j = 1 \mid j^{\text{th}} \text{ received value})}$$

● Normalized Min-Sum Algorithm

The decoder uses the normalized min-sum (NMS) algorithm. The main symbols are defined as follows:

- L_{ch_j} : signed 6-bit channel LLR of the j^{th} VN, with range $[-31, 31]$.
- $L_{c_i \rightarrow v_j}$: signed 6-bit C2V message from the i^{th} CN to the j^{th} VN, with range $[-31, 31]$.
- $L_{v_j \rightarrow c_i}$: signed 6-bit V2C message from the j^{th} VN to the i^{th} CN, with range $[-31, 31]$.
- L_{v_j} : signed 8-bit a posteriori LLR of the j^{th} VN, with range $[-127, 127]$.
- C2V and V2C denote the CN to VN message and VN to CN message respectively.

The decoding procedure consists of the following steps:

(1) Initialization

Set each V2C message $L_{v_j \rightarrow c_i}$ to the corresponding channel LLR L_{ch_j} and initialize each C2V message $L_{c_i \rightarrow v_j}$ to zero.

(2) CN update

For each CN, compute the outgoing C2V message to each neighboring VN using the normalized min-sum rule. The message magnitude is the minimum absolute value of the incoming V2C messages, excluding the target VN. The message sign is the product of the signs of those incoming messages, also excluding the target VN. In this lab, the normalization factor beta is **0.75**. **Round the normalized magnitude to the nearest integer, with halfway cases rounded up, then apply the sign.**

$$L_{c_i \rightarrow v_j} = \prod_{j' \in N(i) \setminus j} \text{sign}(L_{v_{j'} \rightarrow c_i}) \times \beta \cdot \min_{j' \in N(i) \setminus j} (|L_{v_{j'} \rightarrow c_i}|)$$

(3) VN update

For each VN, compute the outgoing V2C message to each neighboring CN by summing the channel LLR and all incoming C2V messages, excluding the target CN. If the result exceeds the signed 6-bit range, **clip it to [-31, 31]**.

$$L_{v_j \rightarrow c_i} = L_{ch_j} + \sum_{i' \in N(j) \setminus i} L_{c_{i'} \rightarrow v_j}$$

A posteriori LLR is computed by summing the channel LLR and all incoming C2V messages from neighboring CNs.

$$L_{v_j} = L_{ch_j} + \sum_{i \in N(j)} L_{c_i \rightarrow v_j}$$

(4) Bit Estimation and Syndrome Check

At the end of each iteration, estimate each codeword bit from the sign of its a posteriori LLR.

$$\hat{x}_j = \begin{cases} 1, & \text{if } L_{v_j} < 0 \\ 0, & \text{otherwise} \end{cases}$$

The syndrome check is used to detect the existence of errors in the decoded sequence. If the syndrome $\mathbf{H}\hat{\mathbf{x}}^T$ is an all-zero vector, the decoded sequence is a valid codeword and decoding stops. Otherwise, the decoder proceeds to the next iteration until the maximum number of iterations is reached.

The matrix multiplication in the syndrome computation is performed over GF(2), i.e., additions are modulo-2 XOR operations.

● Mode 0: Flooding Scheduling

In this lab, $T_{max} = 8$.

```
Initialize all messages;

for ( t = 0; t < Tmax; t = t + 1 ) {

    Perform CN Update for all CNs;
    Perform VN Update for all VNs;

    Perform bit estimation and syndrome check;
    if (syndrome vector is all-zero) {
        break;
    }
}
```

● Mode 1: Layered Scheduling

In this lab, $N_{layer} = 4$ and $T_{max} = 8$.

```

Initialize all messages;

for ( t = 0; t < Tmax; t = t + 1 ) {
    for ( n = 0; n < Nlayer; n = n + 1 ) {
        Perform CN Update for all CNs in Layer n;
        Perform VN Update for all VNs connected to Layer n;
    }

    Perform bit estimation and syndrome check;
    if (syndrome vector is all-zero) {
        break;
    }
}

```

● Output

After decoding, the decoder outputs the 128 final a posteriori LLRs sequentially through **out_data**, from L_{v_0} to $L_{v_{127}}$. The **out_valid** signal must remain high for exactly 128 consecutive cycles, with one a posteriori LLR output per cycle.

If the syndrome check fails after T_{max} iterations, **out_warn** must be high during the entire **out_valid** period. Otherwise, **out_warn** must remain low.

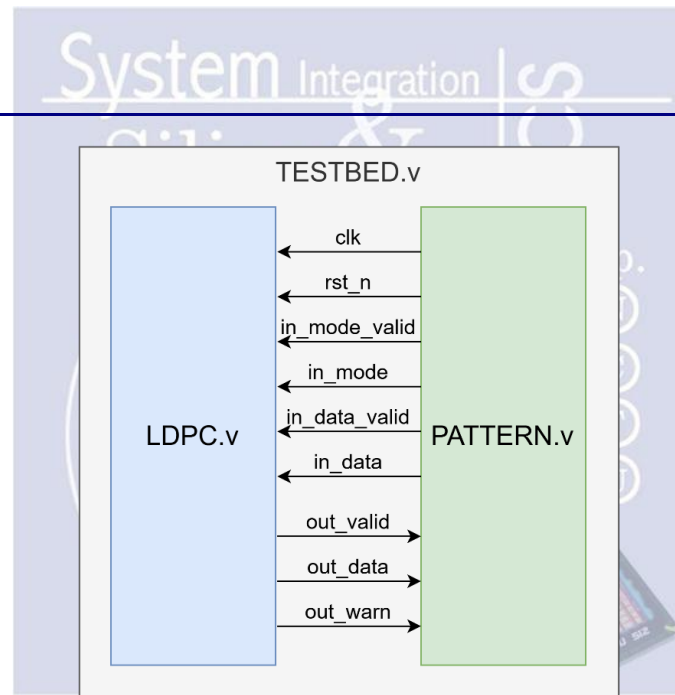
I/O specification

Signals name	Direction	Bit Width	Definition
clk	input	1	Clock.
rst_n	input	1	Asynchronous active-low reset.
in_mode_valid	input	1	High when input in_mode is valid.
in_mode	input	1	0: flooding scheduling. 1: layered scheduling.
in_data_valid	input	1	High when input in_data is valid.
in_data	input	6	Quantized channel LLR input, sent sequentially from L_{ch_0} to $L_{ch_{127}}$. Range: [-31, 31].
out_valid	output	1	High when out_data is valid.
out_data	output	8	Final a posteriori LLR output, sent sequentially from L_{v_0} to $L_{v_{127}}$. Range: [-127, 127]
out_warn	output	1	High during out_valid if the syndrome check fails after T_{max} iterations.

Specifications

1. Top module name: **LDPC** (Filename: **LDPC.v**)
2. It is an **asynchronous** reset and **active-low** architecture. If you use synchronous reset (reset after clock starting) in your design, you may fail to reset signals.
3. The reset signal (**rst_n**) would be given only once at the beginning of simulation. **All output signals should be reset to 0** after the reset signal is asserted.
4. When **out_valid** is low, **out_data** and **out_warn** must be set to **0**.
5. At the beginning of each group of inputs, **in_mode_valid** will be high for one cycle. After **in_mode_valid** is pulled down, the first cycle of **in_data_valid** will occur **2~4** cycles later.
6. The next group of inputs will come in **2~4** cycles after your **out_valid** is pulled down.
7. The **out_valid** should not be high when **in_mode_valid** or **in_data_valid** are high.
8. The total cell area should not larger than **1,500,000 μm^2** .
9. The latency of your design in each pattern should not be larger than **100** cycles. The latency is the clock cycles between the falling edge of the **in_data_valid** and the rising edge of the **out_valid**.
10. The maximum clock period is set to **15ns**, and **you can determine the clock period by yourself**. The precision of clock is 0.1, for example, 14.5 is allowed but 14.55 is not.
11. The input delay is set to **0.5 * (clock period)**.
12. The output delay is set to **0.5 * (clock period)**, and the output loading is set to **0.05**.
13. The synthesis result of data type cannot contain any **LATCH** and **error** (check in syn.log).
14. The synthesis time should be less than **1 hour**.
15. After synthesis, you can check LDPC.area and LDPC.timing. The area report is valid when the slack in the end of timing report should be **non-negative (MET)**.
16. **Timing violations** are **not allowed** in gate-level simulation.
17. **Hardcoding answers** or using a **lookup table** based on the test case index is prohibited; your design must compute outputs correctly based on the actual inputs. You will get 0 point if you use this method, **no second chance**.
18. **Synopsys' IP** and **any comment of // synopsys *** is prohibited in this lab.
19. Only comments in English are allowed.
20. Do not use System Verilog syntax — if your demo fails due to any of the issues, TAs will not be held responsible.
21. Do not use any wire / reg / submodule name containing **“error”**, **“congratulation”**, **“latch”** or **“fail”**, or you will fail the lab.

Block Diagram



Grading Policy

1. Function Validity: 70%
2. Performance: 30%
 - $\text{Cost} = \text{Area}^2 \times \text{Execution time}$
 - $\text{Execution time} = \text{Cycle time} \times \text{Latency}$
 - The less cost your design has, the higher grade you get.

If you fail Lab02 at 1st demo, and pass at **2nd demo**, you will only get **50 point**. Get no score if you fail both 1st and 2nd demo. Note that you will get **0 score** if you are found **plagiarism** at your code.

Note

1. Template folders and reference commands:

01_RTL/	(RTL simulation)	./01_run_vcs_rtl
02_SYN/	(Synthesis)	./01_run_dc_shell
(Check the design if there's latch or not in syn.log)		
(Check the design's timing in /Report/ LDPC.timing)		
03_GATE /	(Gate-level simulation)	./01_run_vcs_gate
09_SUBMIT/(tar all your design)		./00_tar
09_SUBMIT/(submit files)		./01_submit
09_SUBMIT/(check files)		./02_check

2. You should make the clock period values identical in **00_TESTBED/PATTERN.v** and **02_SYN/syn.tcl**

3. In this lab, you can adjust your clock cycle time. Consequently, make sure to key in your clock cycle time after the command like the figure below. It means that the TA will demo your design under this clock cycle time:

```
[~/Lab02 2026F/09 SUBMIT]$ ./00 tar 15.0
```

4. Please submit your files under 09_SUBMIT

- 1st_demo Due day: **09/28 (Mon.) 12:00 pm**
- 2nd_demo Due day: **09/30 (Wed.) 12:00 pm**
- If uploaded files **violate the naming rule**, you will get **5 deduct points**.

5. You should check your submitted files under

09_SUBMIT/Lab02_iclabXXX/ (XXX is your account number)

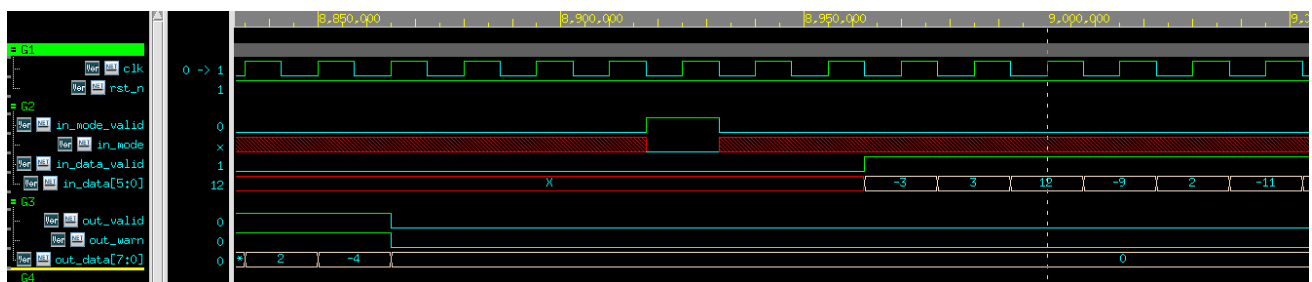
LDPC_iclabXXX.v

filelist_iclabXXX.f

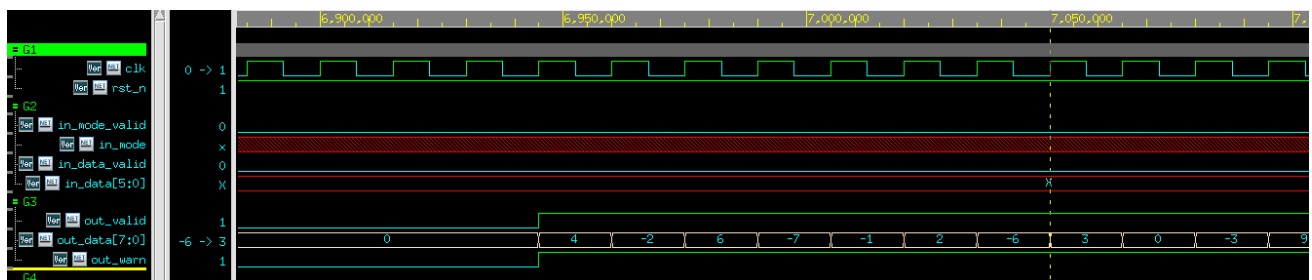
Tclk_iclabXXX.txt (Tclk is your clock period, ex: 15.0_iclab999.txt)

Sample Waveform

Input waveform:



Output waveform:



1. Input Pattern Generation Flow

(1) Information Bits

Randomly generate 64 binary information bits for each pattern. Denote the information row vector by $\mathbf{u} = [\mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{63}]$.

(2) LDPC Encoding

Encode $\mathbf{u}$ with the 64×128 binary generator matrix $\mathbf{G}$ to obtain the 128-bit codeword $\mathbf{x} = \mathbf{u}\mathbf{G}$. All additions are XOR operations. Generator matrix $\mathbf{G}$ is provided in `00_TESTBED/G_128_64.txt`.

(3) BPSK Modulation

For $i = 0, 1, \dots, 127$, map each codeword bit to s_i : bit 0 maps to +1 and bit 1 maps to -1.

(4) AWGN Channel

Generate independent Gaussian noise samples n_i with mean 0 and variance σ^2 , and form the received samples $r_i = s_i + n_i$. $\gamma_{dB} = (\frac{E_b}{N_0})_{dB}$, $\sigma^2 = 10^{-\gamma_{dB}/10}$.

(5) Channel LLR Calculation

Compute the unquantized channel LLR: $L_i = \frac{2r_i}{\sigma^2}$.

(6) Quantization

$L_{chi} = \text{sgn}(L_i) \min(31, \lfloor 2|L_i| + 0.5 \rfloor)$, $i = 0, \dots, 127$.